



Міністерство освіти і науки України
Національний технічний університет України
“Київський політехнічний інститут імені Ігоря Сікорського”
Факультет інформатики та обчислювальної техніки
Кафедра інформаційних систем та технологій

Лабораторна робота №9

із дисципліни «Технології розроблення програмного забезпечення»

Тема: «Взаємодія компонентів системи»

Тема проекту: «Музичний програвач»

GitHub: <https://github.com/martyshenkoo/MusicPlayerWeb>

Виконала:

студентка групи ІА-31:

Мартишенко І.С.

Перевірив:

Мягкий М.Ю.

Мета: Вивчити види взаємодії додатків (Client-Server, Peer-to-Peer, Service-oriented Architecture), та реалізувати в проєктованій системі одну із архітектур. Музичний програвач (iterator, command, memento, facade, visitor, client-server)

Музичний програвач становить собою програму для програвання музичних файлів або відтворення потокової музики з можливістю створення, запам'ятовування і редагування списків програвання, перемішування/повторення (shuffle/repeat), розпізнавання різних аудіо-форматів, еквалайзер.

Завдання

- Ознайомитись з короткими теоретичними відомостями.
- Реалізувати частину функціоналу робочої програми у вигляді класів та їхньої взаємодії для досягнення конкретних функціональних можливостей.
- Реалізувати функціонал для роботи в розподіленому оточенні відповідно до обраної теми.
- Реалізувати взаємодію розподілених частин:
 - *Для клієнт-серверних варіантів:* реалізація клієнтської і серверної частини додатків, а також загальної частини (middleware); зв'язок клієнтської і серверної частин за допомогою WCF, TcpClient, .NET-Remoting на розсуд виконавця.
 - *Для однорангових мереж:* реалізація взаємодії клієнтських додатків за допомогою WCF Peer to peer channel.
 - *Для SOA додатків:* реалізація сервісу, що надає послуги клієнтським застосуванням; викладання сервісу в хмару або підняття у вигляді Web Service на локальній машині; використання токенів для передачі даних про автентифікації, двостороннє шифрування.
- Підготувати звіт щодо виконання лабораторної роботи. Поданий звіт повинен містити: діаграму класів, яка представляє спроектовану архітектуру. Навести фрагменти програмного коду, які є суттєвими для відображення реалізованої архітектури.

Теоретичні відомості

Клієнт-серверні додатки являють собою найпростіший варіант розподілених додатків, де виділяється два види додатків: клієнти (представляють додаток користувачеві) і сервери (використовується для зберігання і обробки даних). Розрізняють тонкі клієнти і товсті клієнти.

Тонкий клієнт – клієнт, який повністю всі операції (або більшість, пов'язаних з логікою роботи програми) передає для обробки на сервер, а сам зберігає лише візуальне уявлення одержуваних від сервера відповідей. Грубо кажучи, тонкий клієнт – набір форм відображення і канал зв'язку з сервером.

Прикладом тонкого клієнта є класичні Web-застосунки.

У такому варіанті використання майже все навантаження лягає на сервер або групу серверів.

Перевагою таких моделей є простота розгортання, тому що оновлювати потрібно лише сервери і в результаті клієнти з наступними запитами автоматично будуть працювати з оновленою системою.

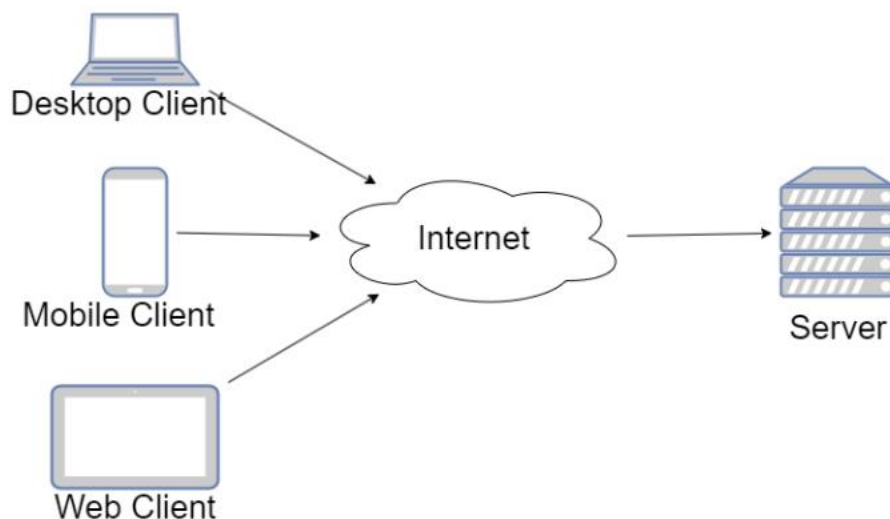


Рис-1. Клієнт-серверна архітектура

Товстий клієнт – антипод тонкого клієнта, більшість логіки обробки даних містить на стороні клієнта. Це сильно розвантажує сервер. Сервер в таких випадках зазвичай працює лише як точка доступу до деякого іншого ресурсу

(наприклад, бази даних) або сполучна ланка з іншими клієнтськими комп'ютерами. Перевагою такого підходу є менші вимоги до серверної частини. Також перевагою, при певному підході до реалізації, є можливість працювати клієнтам без тимчасового доступу до серверу. Прикладом товстого клієнта можна назвати мобільні застосунки, або десктоп застосунки. Наприклад, Evernote, Viber, MS Outlook, комп'ютерні антивіруси, ігри, що потребують інсталяції (The Sims, GTA, ...) та інші.

Проміжним варіантом можна назвати SPA (Single Page Application) – це товсті Web-клієнти, які при старті кожен раз завантажуються з сервера, а надалі працюють з сервером через web-API. З одного боку більшу частину логіки вони відпрацьовують на клієнтській стороні, за рахунок чого зменшується серверне навантаження. Також оновлення простіше ніж для товстих клієнтів. Але такі застосунки не працюють, якщо сервер не доступний.

Клієнт-серверна взаємодія, як правило, організовується за допомогою 3-х рівневої структури: клієнтська частина, загальна частина, серверна частина.

Оскільки велика частина даних загальна (класи, використовувані системою), їх прийнято виносити в загальну частину (middleware) системи.

Клієнтська частина містить візуальне відображення і логіку обробки дії користувача; код для встановлення сеансу зв'язку з сервером і виконання відповідних викликів.

Серверна частина містить основну логіку роботи програми (бізнес-логіку) або ту її частину, яка відповідає зберіганню або обміну даними між клієнтом і сервером або клієнтами.

Хід роботи

У межах проєкту «Музичний програвач» було реалізовано основну логіку роботи з користувачами та аудіотреками. Реалізувати частину функціоналу робочої програми у вигляді класів та їхньої взаємодії для досягнення конкретних функціональних можливостей.

```

62 [HttpPost]
63 public IActionResult Delete(Guid playlistId)
64 {
65     var username = User.Identity!.Name!;
66
67     if (!_history.Backup(username, playlistId))
68     {
69         TempData["Error"] = "Плейліст не знайдено.";
70         return RedirectToAction("Index", "Home");
71     }
72
73     var ok = _playlists.Delete(username, playlistId);
74
75     if (!ok)
76     {
77         TempData["Error"] = "Плейліст не знайдено.";
78         return RedirectToAction("Index", "Home");
79     }
80
81     TempData["Info"] = "Плейліст видалено. Натисни \"Скасувати\", щоб повернути його.";
82     return RedirectToAction("Index", "Home");
83 }
84
85 [HttpPost]
86 public IActionResult Undo(Guid playlistId)
87 {
88     var username = User.Identity!.Name!;
89     if (!_history.Undo(username, playlistId))
90     {
91         TempData["Error"] = "Немає попереднього стану для відновлення.";
92     }
93     else
94     {
95         TempData["Info"] = "Стан плейліста відновлено.";
96     }
97
98     return RedirectToAction("Index", "Home", new { playlistId });
99 }
100
101 [HttpGet]
102 public IActionResult View(Guid id)
103 {
104     var username = User.Identity!.Name!;
105     var playlist = _playlists.GetById(username, id);
106
107     if (playlist == null)
108     {
109         TempData["Error"] = "Плейліст не знайдено.";
110         return RedirectToAction("Index", "Home");
111     }
112
113     var tracks = _playlists.GetTracks(username, id).ToList();
114
115     ViewBag.Playlist = playlist;
116

```

```

117         if (!tracks.Any())
118         {
119             ViewBag.HasIterator = false;
120             return View(tracks);
121         }
122         var iterator = playlist.CreateIterator(tracks);
123
124         ViewBag.HasIterator = true;
125         ViewBag.FirstTrackTitle = iterator.Current().Title;
126         ViewBag.TrackTitlesJson = System.Text.Json.JsonSerializer.Serialize(
127             tracks.Select(t => t.Title).ToList()
128         );
129
130         return View(tracks);
131     }
132 }

```

Рис-2-4. Controllers – PlaylistsController.cs

Services > Visitors > C# PlaylistVisitorService.cs

```

1  using System;
2  using MusicPlayerWeb.Models;
3
4  namespace MusicPlayerWeb.Services.Visitors;
5
6  public class PlaylistVisitorService
7  {
8      private readonly IPlaylistService _playlists;
9
10     public PlaylistVisitorService(IPlaylistService playlists)
11     {
12         _playlists = playlists;
13     }
14
15     public PlaylistStatistics GetStatistics(string username, Guid playlistId)
16     {
17         var playlist = _playlists.GetById(username, playlistId);
18         if (playlist == null)
19             return PlaylistStatistics.Empty;
20
21         var visitor = new PlaylistStatisticsVisitor();
22         playlist.Accept(visitor);
23         return visitor.ToStatistics();
24     }
25 }
26

```

Рис-5. Services – Visitors – PlaylistVisitorService.cs

```

Services > Visitors > C# PlaylistStatisticsVisitor.cs
1  using System.Collections.Generic;
2  using MusicPlayerWeb.Models;
3
4  namespace MusicPlayerWeb.Services.Visitors;
5
6  public class PlaylistStatisticsVisitor : IPlaylistVisitor
7  {
8      private readonly List<string> _titles = new();
9
10     public string? PlaylistTitle { get; private set; }
11     public int TrackCount { get; private set; }
12
13     public void VisitPlaylist(Playlist playlist)
14     {
15         PlaylistTitle = playlist.Title;
16     }
17
18     public void VisitTrack(Track track)
19     {
20         TrackCount++;
21         _titles.Add(track.Title);
22     }
23
24     public PlaylistStatistics ToStatistics()
25     {
26         return new PlaylistStatistics
27         {
28             PlaylistTitle = PlaylistTitle,
29             TrackCount = TrackCount,
30             TrackTitles = _titles.AsReadOnly()
31         };
32     }
33 }

```

Рис-6. Services – Visitors – PlaylistStatisticsVisitor.cs

```

Models > C# Playlist.cs
1  using System.Linq;
2
3  namespace MusicPlayerWeb.Models;
4
5  public class Playlist : IPlaylistVisitable
6  {
7      public Guid Id { get; set; } = Guid.NewGuid();
8      public string Title { get; set; } = default!;
9      public string OwnerUsername { get; set; } = default!;
10     public DateTime CreatedAt { get; set; } = DateTime.UtcNow;
11
12     public ICollection<PlaylistTrack> PlaylistTracks { get; set; } = new List<PlaylistTrack>();
13
14     public ITrackIterator CreateIterator(IEnumerable<Track> tracks)
15     {
16         return new PlaylistIterator(tracks);
17     }
18
19     public void Accept(IPlaylistVisitor visitor)
20     {
21         visitor.VisitPlaylist(this);
22
23         foreach (var track in PlaylistTracks.Select(pt => pt.Track).Where(t => t != null))
24         {
25             track.Accept(visitor);
26         }
27     }
28 }

```

Рис-7. Models – Playlist.cs

```

Models > C# Track.cs
1  namespace MusicPlayerWeb.Models;
2
3  public class Track : IPlaylistVisitable
4  {
5      public Guid Id { get; set; } = Guid.NewGuid();
6      public string Title { get; set; } = default!;
7      public string FileName { get; set; } = default!;
8      public string RelativeUrl { get; set; } = default!;
9      public DateTime AddedAt { get; set; } = DateTime.UtcNow;
10     public string? OwnerUsername { get; set; }
11
12     public void Accept(IPlaylistVisitor visitor)
13     {
14         visitor.VisitTrack(this);
15     }
16 }
17

```

Рис-8. Models – Track.cs

Реалізувати функціонал для роботи в розподіленому оточенні відповідно до обраної теми.

1. Загальна частина

У файлі *Program.cs* налаштовано повний ASP.NET Core пайплайн: *UseRouting*, *UseAuthentication*, *UseAuthorization*, обробка статичних файлів та конфігурація DI-контейнера. У DI реєструються *PlayerClient*, *IPlayerServer*, *PlayerServer*, *PlayerFacade*, а також сервіси роботи з плейлистами. Саме ці *middleware*-компоненти приймають HTTP-запити від браузера та спрямовують їх до відповідних серверних сервісів, забезпечуючи інфраструктуру розподіленого застосунку.

2. Клієнтський рівень

Браузер надсилає HTTP-запити до контролера *PlaylistsController*. Контролер опрацьовує дії користувача: зчитує *User.Identity*, аналізує параметри форм і кнопок, викликає *_playlists*, *_history* та *_playerClient*. Користувацький інтерфейс не взаємодіє напряму з базою даних чи доменними моделями — весь доступ проходить через мережевий рівень та контролер, що відповідає принципам *client-server* архітектури.

3. Канал зв'язку

Компонент *PlayerClient* інкапсулює клієнтські виклики методів *RequestPlay*, *RequestPause*, *RequestStop*. Він не знає деталей реалізації сервера та працює виключно через абстракцію *IPlayerServer*, яка виконує роль контракту між клієнтською частиною та серверною. Такий підхід дозволяє змінювати спосіб комунікації (хоч через WCF, хоч через TCP), хоча наразі взаємодія побудована поверх DI усередині ASP.NET Core.

4. Серверний рівень

PlayerServer реалізує *IPlayerServer* і делегує виконання відповідних операцій у *PlayerFacade*. Саме тут зосереджена бізнес-логіка програвача: керування чергою відтворення, зміною станів, доступом до файлів тощо. Після обробки сервер формує

результат (PlayerOperationResult) і повертає його назад по ланцюжку → до PlayerClient → до контролера → до браузера.

5. Доменний шар

PlayerFacade та сервіси IPlaylistService, ITrackService працюють із даними бази даних і реалізують доменну поведінку застосунку. Цей шар повністю прихований за серверним рівнем: контролери та клієнти не мають до нього прямого доступу, що забезпечує інкапсуляцію та розмежування відповідальностей.

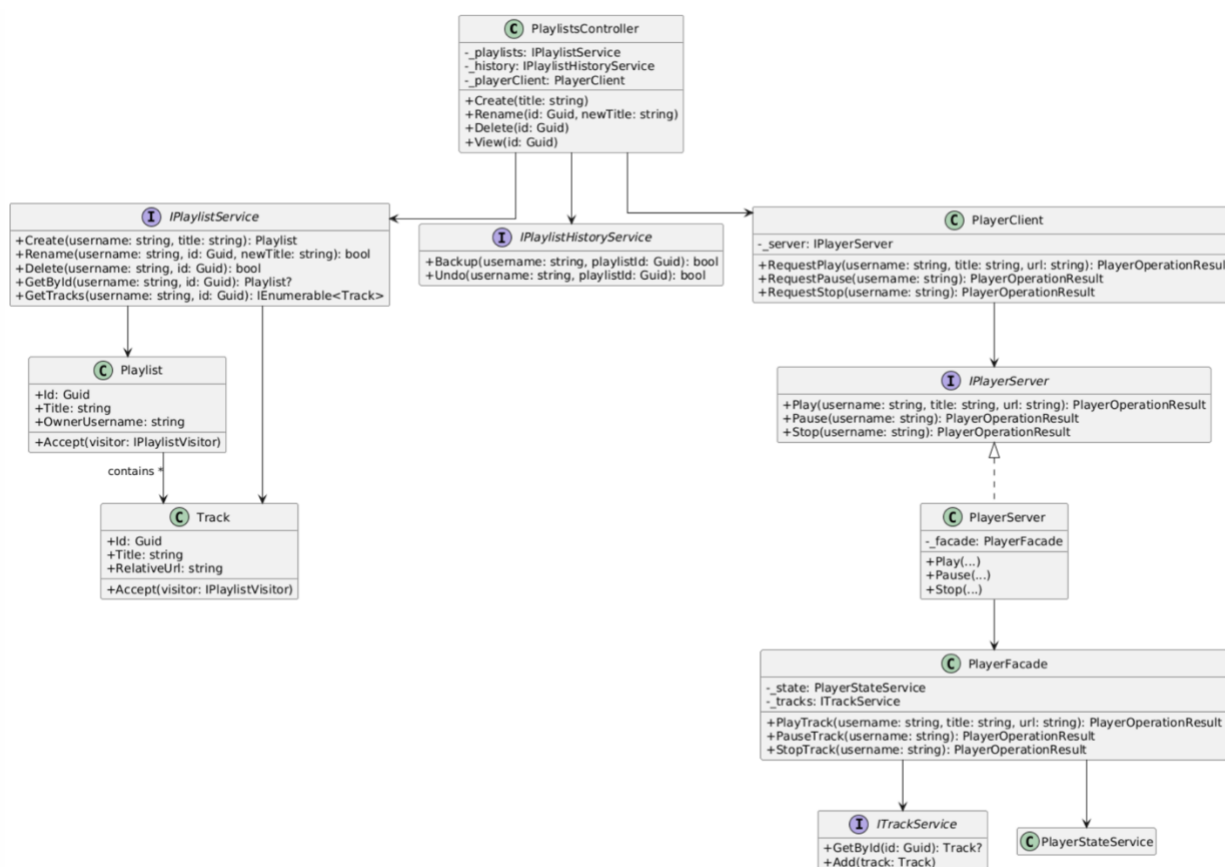


Рис-9. Діаграма класів, що представляє спроектовану архітектуру

1. Контролерний рівень

Клас PlaylistsController отримує HTTP-запити від клієнта та викликає три основні сервіси: IPlaylistService, IPlaylistHistoryService та PlayerClient. Він відповідає за операції створення, перейменування, видалення та перегляду плейлистів. Контролер не працює напряду з базою даних чи доменною логікою - уся взаємодія здійснюється через відповідні сервіси.

2. Сервіси роботи з плейлистами

Інтерфейс `IPlaylistService` забезпечує CRUD-функціональність: створення, перейменування, видалення плейлистів, отримання плейлиста та списку треків.

Інтерфейс `IPlaylistHistoryService` відповідає за історію змін, надаючи методи резервного копіювання та відкату (Backup, Undo).

3. Доменна модель та патерн Visitor

У системі визначено два доменних класи:

- `Playlist` - містить ідентифікатор, назву, власника та набір треків.
- `Track` - містить ідентифікатор, назву та відносний шлях до файлу.

Обидва класи реалізують метод `Accept(visitor)`, що забезпечує підтримку патерну `Visitor`. Це дозволяє додавати нові операції над плейлистами та треками без змін у самих доменних моделях, оскільки поведінка винесена в окремі візитори.

4. Клієнтська частина програвача

Клас `PlayerClient` інкапсулює запити на відтворення, паузу та зупинку (`RequestPlay`, `RequestPause`, `RequestStop`). Він звертається до абстракції `IPlayerServer`, що представляє серверну частину програвача. Таким чином контролер взаємодіє не з логікою програвача напрямку, а через клієнтську обгортку.

5. Сервер програвача

Інтерфейс `IPlayerServer` визначає методи `Play`, `Pause` та `Stop`. Його реалізація - клас `PlayerServer` - приймає виклики від клієнта і делегує їх у `PlayerFacade`, де виконується фактична бізнес-логіка відтворення.

6. Фасад та доменні сервіси програвача

Клас `PlayerFacade` координує роботу двох сервісів:

- `PlayerStateService` = відповідає за поточний стан програвача.
- `ITrackService` - забезпечує доступ до треків (отримання, додавання).

Фасад реалізує методи `PlayTrack`, `PauseTrack` і `StopTrack`, приховуючи внутрішню складність і надаючи єдину точку доступу для сервера.

Реалізувати взаємодію розподілених частин:

Controllers > C# PlaylistsController.cs

```
1  using Microsoft.AspNetCore.Authorization;
2  using Microsoft.AspNetCore.Mvc;
3  using MusicPlayerWeb.Models;
4  using MusicPlayerWeb.Services;
5  using MusicPlayerWeb.Services.Playlists;
6  using System.Linq;
7
8  namespace MusicPlayerWeb.Controllers;
9
10 [Authorize]
11 public class PlaylistsController : Controller
12 {
13     private readonly IPlaylistService _playlists;
14     private readonly IPlaylistHistoryService _history;
15
16     public PlaylistsController(IPlaylistService playlists, IPlaylistHistoryService history)
17     {
18         _playlists = playlists;
19         _history = history;
20     }
21
22     [HttpPost]
23     public IActionResult Create(string title)
24     {
25         var username = User.Identity!.Name!;
26
27         if (string.IsNullOrEmpty(title))
28         {
29             TempData["Error"] = "Назва плейліста є обов'язковою.";
30             return RedirectToAction("Index", "Home");
31         }
32
33         var playlist = _playlists.Create(username, title);
34         return RedirectToAction("Index", "Home", new { playlistId = playlist.Id });
35     }
36
37     [HttpPost]
38     public IActionResult Rename(Guid playlistId, string newTitle)
39     {
40         var username = User.Identity!.Name!;
41
42         if (string.IsNullOrEmpty(newTitle))
43         {
44             TempData["Error"] = "Нова назва не може бути порожньою.";
45             return RedirectToAction("Index", "Home", new { playlistId });
46         }
47
48         if (!_history.Backup(username, playlistId))
49         {
50             TempData["Error"] = "Плейліст не знайдено для створення знімка стану.";
51             return RedirectToAction("Index", "Home", new { playlistId });
52         }
53
54         var ok = _playlists.Rename(username, playlistId, newTitle);
55
56         if (!ok)
57             TempData["Error"] = "Плейліст не знайдено.";
58
59         return RedirectToAction("Index", "Home", new { playlistId });
60     }
61 }
```

```

62 [HttpPost]
63 public IActionResult Delete(Guid playlistId)
64 {
65     var username = User.Identity!.Name!;
66
67     if (!_history.Backup(username, playlistId))
68     {
69         TempData["Error"] = "Плейліст не знайдено.";
70         return RedirectToAction("Index", "Home");
71     }
72
73     var ok = _playlists.Delete(username, playlistId);
74
75     if (!ok)
76     {
77         TempData["Error"] = "Плейліст не знайдено.";
78         return RedirectToAction("Index", "Home");
79     }
80
81     TempData["Info"] = "Плейліст видалено. Натисни \"Скасувати\", щоб повернути його.";
82     return RedirectToAction("Index", "Home");
83 }
84
85 [HttpPost]
86 public IActionResult Undo(Guid playlistId)
87 {
88     var username = User.Identity!.Name!;
89     if (!_history.Undo(username, playlistId))
90     {
91         TempData["Error"] = "Немає попереднього стану для відновлення.";
92     }
93     else
94     {
95         TempData["Info"] = "Стан плейліста відновлено.";
96     }
97
98     return RedirectToAction("Index", "Home", new { playlistId });
99 }
100
101 [HttpGet]
102 public IActionResult View(Guid id)
103 {
104     var username = User.Identity!.Name!;
105     var playlist = _playlists.GetById(username, id);
106
107     if (playlist == null)
108     {
109         TempData["Error"] = "Плейліст не знайдено.";
110         return RedirectToAction("Index", "Home");
111     }
112
113     var tracks = _playlists.GetTracks(username, id).ToList();
114
115     ViewBag.Playlist = playlist;

```

```

117         if (!tracks.Any())
118         {
119             ViewBag.HasIterator = false;
120             return View(tracks);
121         }
122         var iterator = playlist.CreateIterator(tracks);
123
124         ViewBag.HasIterator = true;
125         ViewBag.FirstTrackTitle = iterator.Current().Title;
126         ViewBag.TrackTitlesJson = System.Text.Json.JsonSerializer.Serialize(
127             tracks.Select(t => t.Title).ToList()
128         );
129
130         return View(tracks);
131     }
132 }

```

Рис-10-12. Controllers – PlaylistsController.cs

```

Services > ClientServer > C# PlayerClient.cs
1  using MusicPlayerWeb.Models;
2
3  namespace MusicPlayerWeb.Services.ClientServer;
4
5  public class PlayerClient
6  {
7      private readonly IPlayerServer _server;
8
9      public PlayerClient(IPlayerServer server)
10     {
11         _server = server;
12     }
13
14     public PlayerOperationResult RequestPlay(string username, string title, string url)
15         => _server.Play(username, title, url);
16
17     public PlayerOperationResult RequestPause(string username)
18         => _server.Pause(username);
19
20     public PlayerOperationResult RequestStop(string username)
21         => _server.Stop(username);
22 }

```

Рис-13. Services – ClientServer – PlayerClient.cs

Services > ClientServer > C# IPlayerServer.cs

```
1  using MusicPlayerWeb.Models;
2
3  namespace MusicPlayerWeb.Services.ClientServer;
4
5  public interface IPlayerServer
6  {
7      PlayerOperationResult Play(string username, string title, string url);
8      PlayerOperationResult Pause(string username);
9      PlayerOperationResult Stop(string username);
10 }
11 |
```

Рис-14. Services – ClientServer –IPlayerServer.cs

Services > ClientServer > C# PlayerServer.cs

```
1  using MusicPlayerWeb.Models;
2
3  namespace MusicPlayerWeb.Services.ClientServer;
4
5  public class PlayerServer : IPlayerServer
6  {
7      private readonly PlayerFacade _facade;
8
9      public PlayerServer(PlayerFacade facade)
10     {
11         _facade = facade;
12     }
13
14     public PlayerOperationResult Play(string username, string title, string url)
15     => _facade.PlayTrack(username, title, url);
16
17     public PlayerOperationResult Pause(string username)
18     => _facade.PauseTrack(username);
19
20     public PlayerOperationResult Stop(string username)
21     => _facade.StopTrack(username);
22 }
23 |
```

Рис-15. Services – ClientServer –PlayerServer.cs

C# Program.cs

```
1  using Microsoft.AspNetCore.Authentication.Cookies;
2  using Microsoft.AspNetCore.StaticFiles;
3  using Microsoft.EntityFrameworkCore;
4  using MusicPlayerWeb.Data;
5  using MusicPlayerWeb.Services;
6  using MusicPlayerWeb.Services.Commands;
7  using MusicPlayerWeb.Services.Playlists;
8  using MusicPlayerWeb.Services.ClientServer;
9  using MusicPlayerWeb.Services.Visitors;
10
11  var builder = WebApplication.CreateBuilder(args);
12
13  // MySQL через EF Core
14  var connectionString = builder.Configuration.GetConnectionString("DefaultConnection");
15  builder.Services.AddDbContext<AppDbContext>(options =>
16  |   options.UseMySQL(connectionString, ServerVersion.AutoDetect(connectionString)));
17
18  // Auth (cookies)
19  builder.Services.AddAuthentication(CookieAuthenticationDefaults.AuthenticationScheme)
20  |   .AddCookie(options =>
21  |   {
22  |       options.LoginPath = "/Account/Login";
23  |       options.LogoutPath = "/Account/Logout";
24  |       options.AccessDeniedPath = "/Account/Login";
25  |   });
26
27  builder.Services.AddControllersWithViews();
28  builder.Services.AddMemoryCache();
29
30  // DI: Db-сервіси
31  builder.Services.AddScoped<IUserService, DbUserService>();
32  builder.Services.AddScoped<ITrackService, DbTrackService>();
33  builder.Services.AddScoped<IPlaylistService, DbPlaylistService>();
34  builder.Services.AddScoped<IPlaylistHistoryService, PlaylistHistoryService>();
35  builder.Services.AddScoped<PlayerFacade>();
36  builder.Services.AddScoped<IPlayerServer, PlayerServer>();
37  builder.Services.AddScoped<PlayerClient>();
38  builder.Services.AddScoped<PlaylistVisitorService>();
39  builder.Services.AddSingleton<PlayerStateService>();
40  builder.Services.AddSingleton<PlayerCommandInvoker>();
41
42  builder.Services.AddHttpContextAccessor();
43  var app = builder.Build();
44
45  // статичні файли + коректні MIME для аудіо
46  var provider = new FileExtensionContentTypeProvider();
47  provider.Mappings[".m4a"] = "audio/mp4";
48  provider.Mappings[".flac"] = "audio/flac";
49  app.UseStaticFiles(new StaticFileOptions { ContentTypeProvider = provider });
50
51  if (!app.Environment.IsDevelopment())
52  |   {
53  |       app.UseExceptionHandler("/Home/Error");
54  |       app.UseHsts();
55  |   }
56  app.UseHttpsRedirection();
57  app.UseRouting();
58
59  app.UseAuthentication();
60  app.UseAuthorization();
61
62  app.MapControllerRoute(
63  |   name: "default",
64  |   pattern: "{controller=Home}/{action=Index}/{id?}");
65
66  var uploadsPath = Path.Combine(app.Environment.WebRootPath, "uploads");
67  Directory.CreateDirectory(uploadsPath);
68
69  app.Run();
70
```

Рис-16. Program.cs

Controllers/PlaylistsController.cs

Контролер на серверній стороні (ASP.NET Core) приймає HTTP-запити від браузера та працює з сервісами `IPlaylistService` і `IPlaylistHistoryService`. Він повертає результати клієнту у вигляді HTTP-відповідей, при цьому інтерфейс користувача не має прямого доступу до даних. Увесь обмін відбувається виключно через мережевий контролер, що відповідає класичній client-server моделі.

Services/ClientServer/PlayerClient.cs

Цей компонент виступає внутрішнім клієнтом у системі. Він інкапсулює виклики до функціональності програвача, тож контролер взаємодіє саме з ним. Логіка відтворення музики проходить через зв'язку "клієнт → сервер", що підкреслює побудову застосунку як розподіленого.

Services/ClientServer/IPlayerServer.cs та PlayerServer.cs

Інтерфейс `IPlayerServer` описує контракт сервера для роботи з програвачем. `PlayerClient` викликає його методи, а реалізація `PlayerServer` делегує потрібні операції у `PlayerFacade` та відповідні доменні сервіси. Це забезпечує відокремлення мережевої взаємодії від бізнес-логіки.

Program.cs

Файл містить налаштування middleware ASP.NET Core - маршрутизації, автентифікації, залежностей та іншої інфраструктури, що забезпечує роботу розподіленого застосунку. Уся система працює як розподілене середовище: дії користувача з браузера потрапляють до серверного контролера, далі через `PlayerClient` надсилаються до серверної частини програвача, а виконання завдань координує інфраструктура ASP.NET Core. Це забезпечує чітке розділення відповідальностей і дотримання архітектурної моделі client-server.

Висновок:

У межах цієї лабораторної роботи я побудувала клієнт-серверну архітектуру веб-програвача `MusicPlayerWeb`. Серверна частина, реалізована на ASP.NET Core,

відповідає за обробку HTTP-запитів, керування станом програвача та виконання бізнес-логіки. До цього шару належать контролери, сервіси роботи з плейлистами, а також компоненти PlayerServer і PlayerFacade, що інкапсулюють логіку відтворення. Окремо реалізовано механізм збору статистики плейлистів за допомогою патерну Visitor, що дає змогу розширювати функціональність без модифікації доменної моделі. Клієнтська частина представлена «тонким» інтерфейсом на Razor-сторінках: вона містить форми створення та редагування плейлистів, відображення треків і елементи керування відтворенням, які лише надсилають запити на сервер. Уся логіка знаходиться на серверному боці, що відповідає принципам розподіленого застосунку. Посередницький шар - залежності, налаштовані через DI та middleware у Program.cs, разом із PlayerClient, IPlayerServer і доменними сервісами - забезпечує слабе зв'язування між UI та бізнес-логікою. Завдяки цьому клієнт взаємодіє з сервером через чітко визначені контракти, а внутрішні архітектурні рішення, такі як Visitor для статистики та Facade для роботи програвача, демонструють практичне застосування патернів проєктування. У результаті отримано функціональну розподілену систему з прозорими інтерфейсами взаємодії, чітким поділом відповідальностей і гнучкою архітектурою, яку можна розширювати без суттєвих змін у базових компонентах.

Відповіді на контрольні питання

1. Що таке клієнт-серверна архітектура?

Клієнт-серверна архітектура - це модель організації комп'ютерних систем, де клієнти (зазвичай користувацькі програми або пристрої) роблять запити на сервер, який обробляє ці запити і повертає результати. Клієнт відповідає за інтерфейс користувача та ініціацію запитів, сервер - за обробку даних, зберігання та логіку.

2. Розкажіть про сервіс-орієнтовану архітектуру (SOA).

SOA - це архітектурний підхід, де функціональність програми реалізована у вигляді сервісів - автономних компонентів, що виконують конкретні бізнесзавдання. Кожен сервіс має чіткий інтерфейс і взаємодіє з іншими через стандартизовані протоколи (наприклад, HTTP, SOAP, REST).

3. Якими принципами керується SOA?

Основні принципи SOA:

- Автономність сервісів - сервіси працюють незалежно.
- Стандартизовані інтерфейси - сервіси взаємодіють через визначені API.
- Повторне використання - сервіси можна використовувати в різних системах.
- Легко інтегрувати - сервіси повинні легко поєднуватись у складні процеси.
- Слабке зв'язування (loose coupling) - зміни в одному сервісі мінімально впливають на інші.

4. Як між собою взаємодіють сервіси в SOA?

Сервіси взаємодіють через стандартизовані повідомлення або API, наприклад через SOAP або REST. Кожен сервіс публікує свій інтерфейс (WSDL, OpenAPI), і інші сервіси можуть викликати його методи, обмінюючись даними у формі XML, JSON або інших форматах.

5. Як розробники дізнаються про існуючі сервіси і як робити до них запити?

- Реєстр сервісів (Service Registry) - централізована база, де зареєстровані всі сервіси та їхні інтерфейси.
- Документація API (наприклад OpenAPI/Swagger).
- Запити здійснюються через стандартні протоколи (HTTP, SOAP, REST) за адресою сервісу і з використанням описаних методів та форматів даних.

5. У чому полягають переваги та недоліки клієнт-серверної моделі?

Переваги:

- Центральне зберігання даних, легший контроль безпеки.
- Легко масштабувати сервери.
- Клієнти можуть бути простими, вся логіка на сервері.

Недоліки:

- Сервер може стати вузьким місцем (single point of failure).
- Високі вимоги до потужності сервера при великій кількості клієнтів.
- Залежність клієнта від сервера - без доступу до сервера система не працює.

7. У чому полягають переваги та недоліки однорангової (peer-to-peer) моделі взаємодії?

Переваги:

- Відсутність централізованого сервера, підвищена стійкість до відмов.

- Можливість прямого обміну ресурсами між учасниками.
- Масштабування «горизонтальне» - додаючи вузли, підвищуєш потужність.

Недоліки:

- Складніше забезпечувати безпеку та контроль доступу.
- Кожен вузол відповідає за управління ресурсами.
- Важче координувати оновлення і синхронізацію даних.

8. Що таке мікросервісна архітектура? Мікросервісна архітектура — це підхід, де додаток складається з малих, незалежних сервісів, кожен з яких реалізує одну бізнес-функцію і може розгортатися окремо. Вона є розвитком SOA, але з більш дрібними і автономними компонентами.

9. Які протоколи використовуються для обміну даними в мікросервісній архітектурі?

- HTTP/HTTPS + REST
- gRPC
- SOAP
- Message brokers: RabbitMQ, Kafka, MQTT для асинхронної взаємодії
- WebSockets для реального часу

10. Чи можна назвати підхід сервіс-орієнтованою архітектурою, якщо у проєкті між веб-контролерами та шаром доступу до даних реалізуємо шар бізнес-логіки у вигляді сервісів?

Це не повноцінна SOA, а локальне використання сервісів всередині одного додатку. SOA передбачає автономні, незалежні сервіси, доступні через стандартизовані протоколи для інших систем. Твій підхід — це архітектурний патерн «сервісний шар» (Service Layer), який організовує бізнес-логіку всередині одного проєкту.